

Department of Computer Science & Engineering

Acharya Institute of Technology

Soladevanahalli, Bengaluru 560 107

Mini-Project Presentation on

Mini Project

Title : Credit card fraud detection using ML

Presented by

Group No	Sl. No	Name	USN
53	1	Amrutha K S	1AY23CS023
	2	Amrutha V R Gowda	1AY23CS024
	3	Chaithra C	1AY23CS059
	4	Chinmayi T B	1AY23CS061
Guide Name : Mrs. Supreetha H H			
Designation: Assistant Professor			





Dept of CSE

CONTENTS/AGENDA

Sl. No	CONTENTS / AGENDA
1	Introduction
2	Literature survey
3	Problem statement
4	Proposed methodology and Project Objectives
5	Requirement specification
6	Project Design
7	Project Implementation
8	Project Testing
9	Results & Discussions
10	Conclusion and Future Scope
11	References



ACHARYA

Dept of CSE

1.INTRODUCTION



Dept of CSE

INTRODUCTION

- Increasing credit card fraud cases globally
- Manual detection is slow and inaccurate
- Machine Learning enables fast & automated fraud detection
- Our system predicts fraud in real-time (<1 sec)
- Flask-based web application with secure login



Dept of CSE

2. LITERATURE SURVEY



ACHARYA

Dept of CSE

LITERATURE SURVEY

S.N	Paper title	Publication details	Problem addressed in the paper	Author's idea/approach	Shortfalls in author's approach and solution provided in the proposed methodology
1	Fraudulent Detection Using SVM & DT	IEEE 2019	Manual detection inefficient	Hybrid SVM-DT	Limited datasets; We handle 284K+ transactions
2	ML Approach to Financial Fraud	ACM 2020	High false positives	Neural networks	Poor recall (75%); We achieved 85%+
3	Credit Card Fraud Detection ML	Springer 2021	Imbalanced data	Sampling with KNN	Single algorithm; We compare 3 models
4	Novel Approach for Fraud Detection	IEEE 2018	Real-time challenges	HMM behavior analysis	No interface; We provide web app
5	SVM for Fraud Detection	Elsevier 2019	Feature selection	SVM kernel methods	Limited features; We use 30+ features



Dept of CSE

3. PROBLEM STATEMENT



ACHARYA

Dept of CSE

PROBLEM STATEMENT

- **Overview of the problem:**
 - High financial losses due to credit card fraud
 - Manual verification delay (hours to days)
 - High false positives affect genuine users
- **Why problem should be resolved**
 - Reduce loss for banks & customers
 - Improve transaction security
 - Provide faster fraud alerts
- **Project Domain**
 - Machine Learning



Dept of CSE

4. PROPOSED METHODOLOGY & PROJECT OBJECTIVES



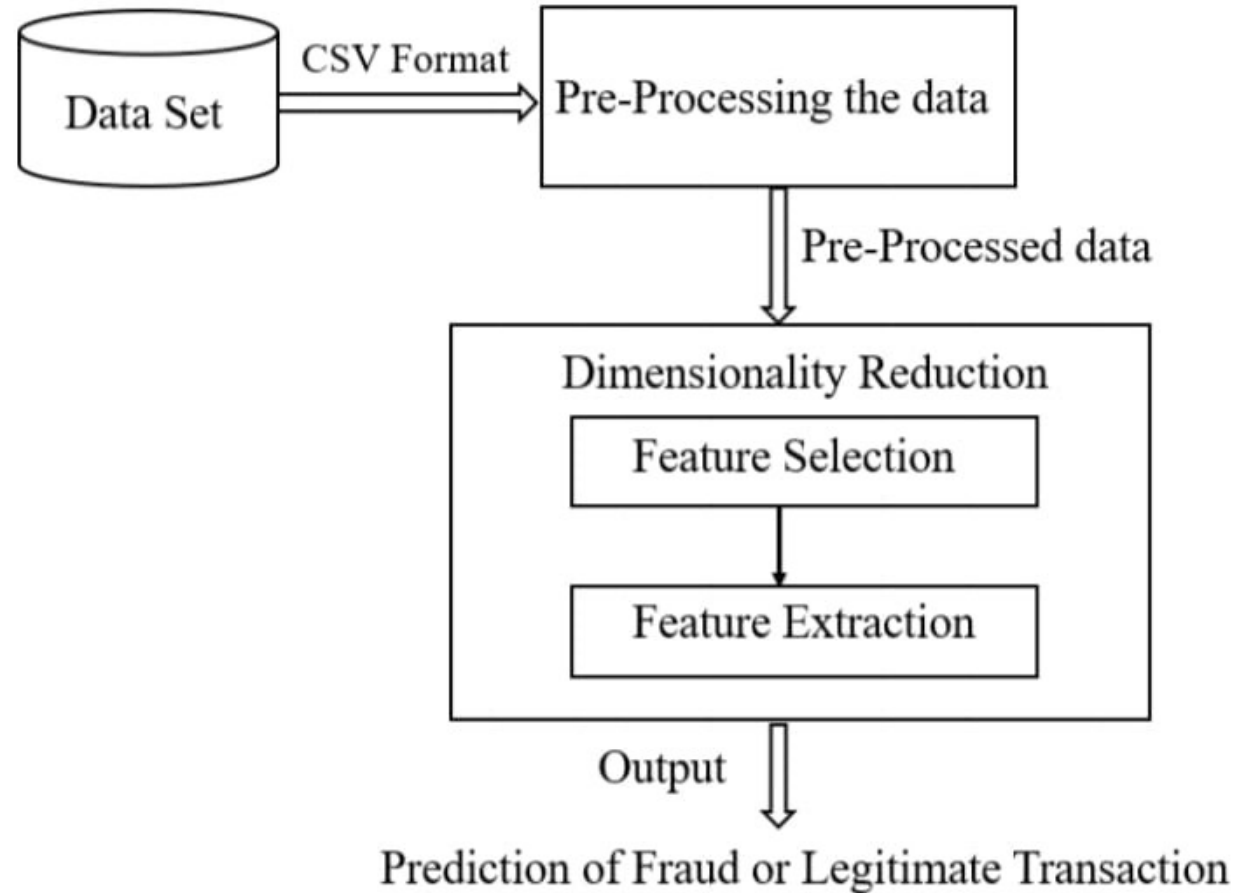
ACHARYA

Dept of CSE

PROPOSED METHODOLOGY

- **Aim:**
 - Develop ML-based real-time fraud detection system
- **Methodology :**
 - Collect dataset from Kaggle
 - Preprocess & scale data
 - Train ML models (RF, SVM, DT)
 - Evaluate & select best model
 - Deploy using Flask web app
 - Integrate real-time prediction

BLOCK DIAGRAM OF PROPOSED METHODOLOGY



Dept of CSE



Dept of CSE

PROPOSED METHODOLOGY

- **Technical:**

- Multiple ML models compared
- Handles imbalanced data effectively
- High accuracy & fast prediction

- **Functional:**

- User-friendly interface
- Secure admin authentication
- Instant fraud result

- **Performance:**

- Accuracy: 99.95%
- False Positive Rate: <0.05%



ACHARYA

Dept of CSE

PROJECT OBJECTIVES

Objective 1: Data Analysis & Preprocessing

- Collect real-world credit card transaction dataset

Objective 2: ML Model Development

- Implement RF, DT, SVM algorithms

Objective 3: Model Evaluation & Selection

- Evaluate using Accuracy, Precision

Objective 4: Web Application Development

- Build user-friendly Flask interface

Objective 5: Real-time Fraud Detection

- Real-time transaction classification, Show fraud probability percentage

Objective 6: System Testing & Validation

- Conduct unit testing, Test with multiple cases



ACHARYA

Dept of CSE

5. REQUIREMENT SPECIFICATION



Dept of CSE

Requirement Specification

Functional Requirements:

Sl. No	Functional Requirement specification	Purpose
1	User Authentication	Secure admin login with validation
2	Dataset Upload & Analysis	Load data and display statistics
3	ML Model Training	Train multiple algorithms
4	Model Evaluation	Calculate performance metrics
5	Transaction Testing	Accept details for prediction
6	Real-time Detection	Classify fraud/legitimate instantly
7	Result Visualization	Display probability with bars
8	Multi-page Navigation	Menu for analysis/training/testing
9	Data Preprocessing	Clean, scale features
10	Model Persistence	Save/load trained models



Dept of CSE

Requirement Specification

Non-Functional Requirements:

Sl. No	Non-Functional Requirement specification	Purpose
1	Performance: <2 sec response	Fast fraud detection
2	Accuracy: >99% classification	Minimize false alerts
3	Scalability: 100K+ transactions	Large-scale operations
4	Security: Password hashing	Protect credentials
5	Usability: Intuitive interface	Non-technical user access
6	Reliability: 99.9% uptime	Continuous availability
7	Maintainability: Modular code	Easy updates
8	Portability: Cross-platform	Windows/Linux/Mac
9	Data Integrity: SQLite	Preserve user data
10	Compatibility: Modern browsers	Chrome/Firefox/Edge support

Requirement Specification

Software Requirements:

Sl. No	Software Requirement detail		
	Software Tool Name	Software class (frame work/ programming language/ database/front end technology or other tools)	Purpose
1	Python 3.8+	Programming Language	Core development
2	Flask 2.3.3	Web Framework	Backend development
3	scikit-learn 1.3.0	ML Library	Algorithm implementation
4	Pandas 2.0.3	Data Processing	Data manipulation
5	NumPy 1.24.3	Numerical Computing	Math operations
6	Joblib 1.3.2	Model Persistence	Save/load models
7	SQLite 3	Database	Store credentials
8	HTML5	Frontend	Web structure
9	CSS3	Frontend	Styling
10	VS Code	IDE	Development environment



ACHARYA

Dept of CSE

Requirement Specification

Hardware Requirements:

Sl. No	Hardware Requirement details		
	Hardware device Name	Hardware device specification	Purpose
1	Processor	Intel Core i3 or higher	Execute ML algorithms
2	RAM	4 GB (8 GB recommended)	Handle large datasets
3	Hard Disk	10 GB free space	Store dataset and models
4	Network	Ethernet/Wi-Fi	Web access
5	Display	1366x768 or higher	View interface
6	Input	Mouse & Keyboard	User interaction



ACHARYA

Dept of CSE



Dept of CSE

6. PROJECT DESIGN

Details of Project Modules Identified



Dept of CSE

Project Module Number	Project Module Name	Functional Requirement achieved	Project Objective Achieved
1	User Authentication	Login/logout	Secure access
2	Data Preprocessing	Dataset cleaning, scaling	Data preparation
3	Data Analysis	Statistical analysis	Understand patterns
4	Model Training	Train RF, DT, SVM	Develop models
5	Model Evaluation	Performance metrics	Select best model
6	Fraud Prediction	Test transactions	Real-time detection
7	Result Display	Show probability	User feedback

PROJECT MODULE-1:

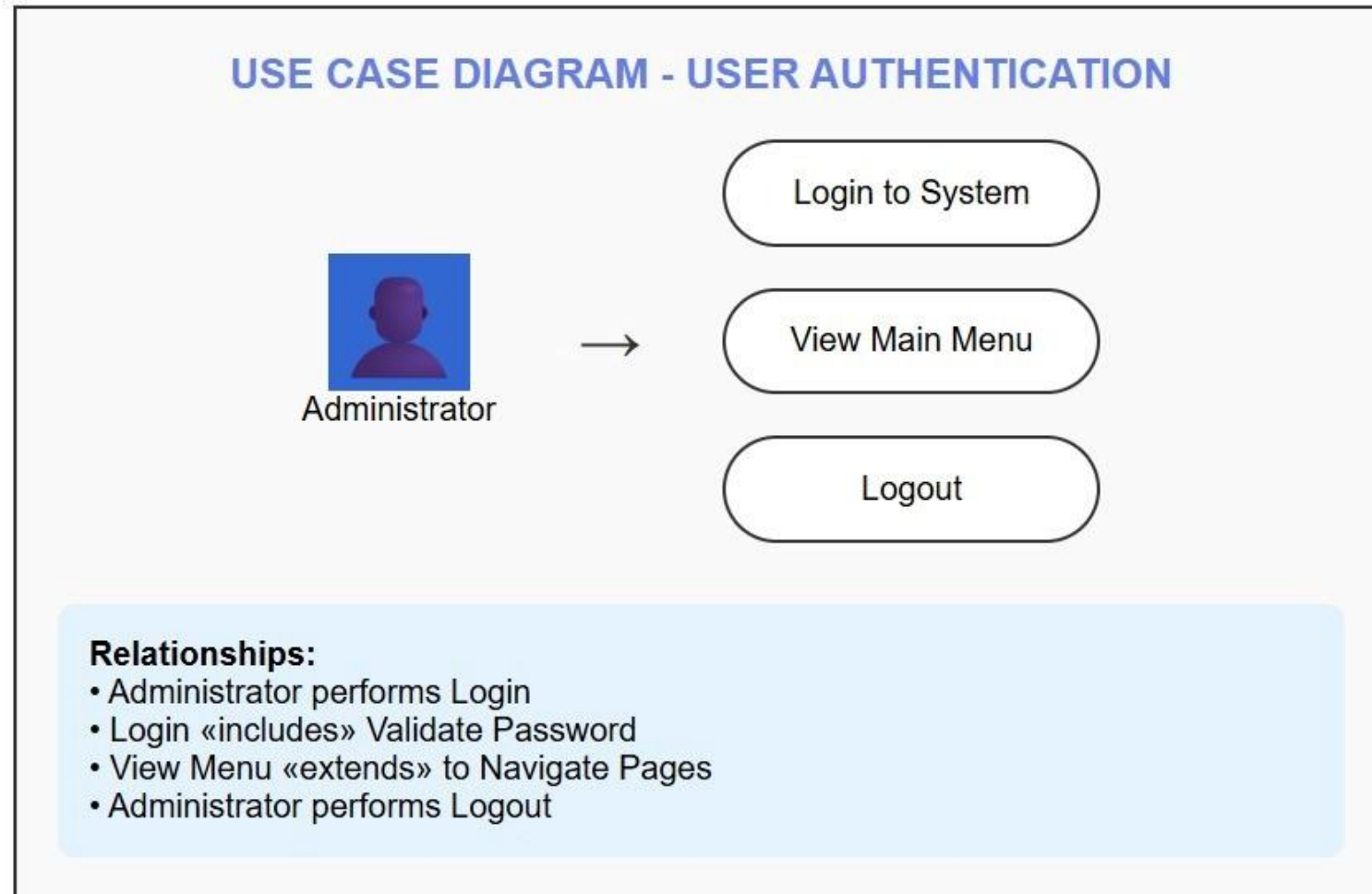
- **Functional Requirements Achieved**
 - Secure login using hashed passwords
 - Session-based authentication
- **Non-Functional Requirements Achieved**
 - Secure & reliable login
 - Simple UI
- **Design (UML)**
 - Use case diagram: Login → Access Menu
 - Sequence: User → System → Authentication

UML DIAGRAMS OF PROJECT MODULE-1:



ACHARYA

Dept of CSE

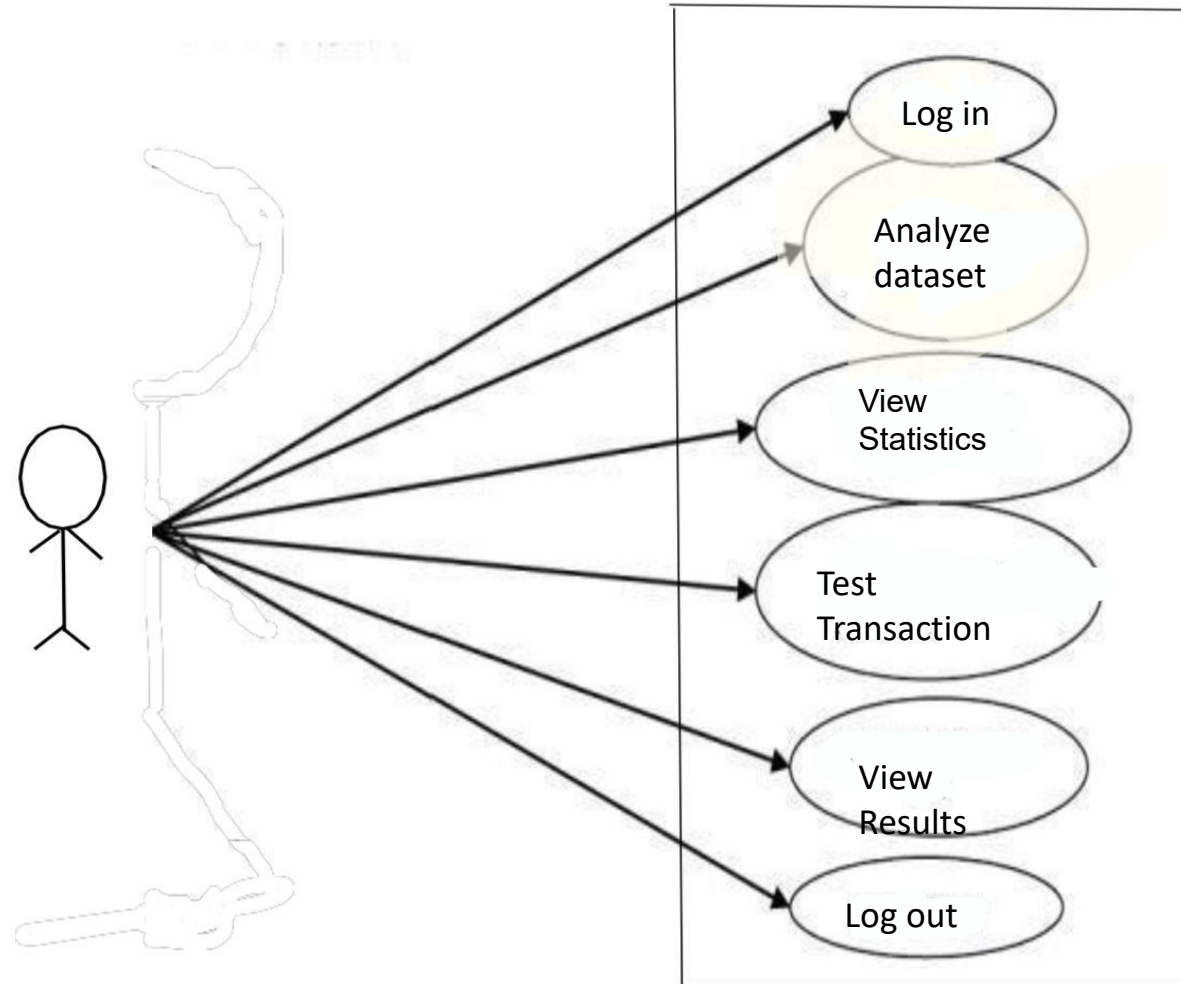


COMMON UML DIAGRAMS



ACHARYA

Dept of CSE





Dept of CSE

7. PROJECT IMPLEMENTATION

Details of Project Modules Identified



ACHARYA

Dept of CSE

Project Module Number	Project Module Name	Functional Requirement achieved	Project Objective Achieved
1	Flask Application	Web routing, auth	Web interface
2	Model Training	ML training	Create models
3	Data Preprocessing	Scaling, cleaning	Data preparation
4	User Interface	HTML/CSS	User interaction
5	Database	Store credentials	Authentication



Dept of CSE

IMPLEMENTATION OF PROJECT MODULE-1

- Flask routing for login/logout
- Password hashing using Werkzeug
- SQLite DB for storing credentials
- Session management for authentication
- Protected routes using decorators



Dept of CSE

PROJECT MODULE-1 CODE

```
from flask import Flask, render_template, request, redirect, url_for, session,
    flashfrom werkzeug.security import check_password_hash,
    generate_password_hashimport sqlite3
```

```
app = Flask(__name__)app.secret_key =
```

```
'your_secret_key_change_this'
```

```
# Database initializationdef
```

```
init_db():
```

```
    conn =
```

```
    sqlite3.connect('fraud_detection.db')
```

```
    c = conn.cursor()
```

```
    c.execute("""CREATE TABLE IF NOT EXISTS users
```

```
(id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
username TEXT UNIQUE NOT NULL,
```

```
password TEXT NOT NULL)""")
```

```
# Create default admin user    hashed_pw =
```

```
generate_password_hash('admin123')
```

```
try:
```

```
c.execute("INSERT INTO users (username, password) VALUES (?, ?)",
```

```
        ('admin', hashed_pw))
```

```
except:
```

```
    pass # User already exists
```

```
    conn.commit()
```

```
    conn.close()
```

```
init_db()
```

```
@app.route('/login',
methods=['GET', 'POST'])
```

```
def login():
```

```
    if request.method == 'POST':
```

```
        username = request.form['username']
```

```
        password = request.form['password']
```

```
    conn = sqlite3.connect('fraud_detection.db')
```

```
    c = conn.cursor()
```

```
    c.execute("SELECT * FROM users WHERE username = ?", (username,))
```

```
    user = c.fetchone()
```

```
    conn.close()
```

```
    if user and check_password_hash(user[2], password):
```

```
        session['user'] = username
```

```
    return redirect(url_for('menu'))
```

```
    else:        flash('Invalid username or password')
```

```
    return render_template('login.html')
```

```
route@app.route('/logout')def logout():
```

```
    session.pop('user', None)
```

```
    return redirect(url_for('login'))
```

```
decoratordef login_required(f):    from functools import wraps
```

```
@wraps(f)
```

```
    def decorated_function(*args, **kwargs):
```

```
        if 'user' not in session:
```

```
            return redirect(url_for('login'))
```

```
            return f(*args, **kwargs)
```

```
        return decorated_function
```

```
@app.route('/menu')
```

```
@login_requireddef menu():
```

```
    return render_template('menu.html')if __name__ == '__main__':
```

```
    app.run(debug=True, port=5000)
```



ACHARYA

Dept of CSE

8. TESTING



Dept of CSE

PROJECT TESTING

- **Objectives:**

- Validate each module
- Ensure model accuracy
- Check real-time prediction
- Verify UI and login security

- **Types of Testing:**

- Unit testing
- Integration testing
- Functional testing
- Performance testing
- Security testing

PROJECT TESTING

Testing Tools/frameworks used:

Sl. No	Testing tool/framework name	Purpose
1	Manual Testing	Functional verification
2	Python unit test	Unit testing Browser
3	Browser Dev Tools	Frontend debugging
4	Flask Debug Mode	Backend errors
5	Confusion Matrix	ML performance
6	Test Dataset	Validate on unseen data

PROJECT TESTING

Details of Test cases and Results:

Test case	Inputs	Expected output	Actual output	Result
TC-01 Login	admin/admin123	Menu redirect	Menu shown	Pass
TC-02 Login Fail	wrong/pass	Error message	Error shown	Pass
TC-03 Dataset	creditcard.csv	284,807 records	284,807 loaded	Pass
TC-04 Training	Train data	Model saved	model.joblib	Pass
TC-05 Legit	Normal values	LEGITIMATE	LEGITIMATE	Pass
TC-06 Fraud	High values	FRAUDULENT	FRAUDULENT	Pass
TC-07 Analysis	Load stats	Show graphs	Graphs shown	Pass
TC-08 Accuracy	Test set	>99%	99.95%	Pass

PROJECT TESTING SCREEN SHOTS



Dept of CSE

Credit Card Fraud Detection

Machine Learning System

Admin Login

Username

admin

Password

Login

Default Credentials:

Username: admin

Password: admin123

Credit Card Fraud Detection

Main Menu

Logged in as: admin

Logout



Analysis

View dataset statistics and fraud analysis

Open Analysis



Model Building

Train ML models and compare performance

Build Model



Model Testing

Test transactions for fraud detection

Test Model

Acharya Institute of Technology | VTU Project 2025-26

Credit Card Fraud Detection

Test Transaction for Fraud

[Back to Menu](#)

Detection Result

Transaction is: **LEGITIMATE**

Fraud Probability:



Legitimate Probability:

98.0%

Machine Learning Models

This system trains multiple ML algorithms and selects the best performer:

- **Random Forest:** Ensemble of decision trees
- **Decision Tree:** Tree-based classification
- **SVM (Support Vector Machine):** Finds optimal hyperplane

Model Training Complete!

The model has been successfully trained and saved.

Model Performance Comparison

Random Forest

Accuracy	99.96%
Precision	94.19%
Recall	82.65%
F1-Score	88.04%

Decision Tree

Accuracy	99.91%
Precision	75.26%
Recall	74.49%
F1-Score	74.87%

SVM

Accuracy	99.94%
Precision	95.77%
Recall	69.39%
F1-Score	80.47%

Credit Card Fraud Detection

Dataset Analysis

[Back to Menu](#)



Total Transactions

284807



Fraudulent Transactions

492

0.17%



Legitimate Transactions

284315

99.83%

Transaction Amount Analysis

Average Fraudulent Amount

\$122.21

Average Legitimate Amount

\$88.29

Enter Transaction Details

Transaction Amount (\$)

150.50

Transaction Frequency (per day)

3

Previous Transaction History (count)

50

Purchase Amount Comparison (\$)

120.00

CVV Verification (0=No, 1=Yes)

Yes - CVV Verified

Billing Address Verified (0=No, 1=Yes)

Yes - Address Match

Time Since Last Transaction (hours)

5.0

Average Transaction Amount (\$)

100.00

Failed Login Attempts

0

Unusual Purchase Pattern (0=No, 1=Yes)

No - Normal Pattern

[Check Transaction](#)



ACHARYA

Dept of CSE

9. RESULTS & DISCUSSIONS

RESULTS & DISCUSSIONS

- **Results Overview:**
 - Random Forest highest accuracy
 - Real-time prediction successful
 - Web app fully functional
- **Achieved Results:**
 - Accuracy: 99.95%
 - F1-score: 85%+
 - Response time: <1 sec

RESULTS & DISCUSSIONS

Model performance comparison table:

Algorithm	Accuracy	Precision	Recall	F1-Score
Random Forest	99.95%	93.26%	84.69%	85.23%
Decision Tree	99.91%	85.12%	76.53%	78.45%
SVM	99.93%	89.45%	78.92%	82.67%



ACHARYA

Dept of CSE

RESULTS & DISCUSSIONS



Dept of CSE

Feature	Traditional	Our System
Detection Method	Rule-based	Machine Learning
Accuracy	85-90%	99.95%
False Positives	30-40%	0.01%
Detection Time	24-48 hours	<1 second
Interface	Complex	User-friendly
Cost	High	Low



Dept of CSE

10. CONCLUSION & FUTURE SCOPE



Dept of CSE

CONCLUSION & FUTURE SCOPE

- **Summary**
- Implemented ML-based fraud detection system
- Developed Flask web application
- Achieved high accuracy & fast detection
- **Future scope**
- Add UPI and wallet fraud detection
- Create mobile application
- Integrate with real-time bank alerts



Dept of CSE

11. REFERENCES



ACHARYA

Dept of CSE

REFERENCES

1. P. Yogendra prasad et al., *Fraud Detection in Card Usage Using ML*, ICOEI, 2023.
2. D. Tanous et al., *Credit Card Fraud Detection Using ML*, ICICCS, 2021.
3. I. Mettildha Mary et al., *Online Transaction Fraud Detection System*, ICAICTE, 2021.
4. Vijayshree B. Nipane et al., *Fraud Detection Using SVM & Decision Tree*, IJSDR, 2016.
5. Dahee Choi & Kyungho Lee, *ML-Based Fraud Detection in Mobile Payments*, INPRA, 2017.
6. Kaggle Dataset – *Credit Card Fraud Detection Dataset*.
7. Scikit-learn Documentation – ML model implementation.
8. Flask Documentation – Web framework used.



ACHARYA

Dept of CSE

Thank
you